

# Mini Guida Pratica: L'Ereditarietà in C++

Istruzioni, parole chiave e gestione dei livelli di accesso (public, protected, private)

L'**Ereditarietà** è il pilastro della programmazione a oggetti che permette a una nuova classe (classe **Figlia** o *Derivata*) di ereditare gli attributi e i metodi di una classe esistente (classe **Padre** o *Base*). Serve a massimizzare il riutilizzo del codice.

## 1. La Sintassi di Base e il Trabocchetto `public`

Per far ereditare una classe, si usano i due punti ( `:` ) seguiti dalla parola chiave `public` e dal nome della classe padre.

```
class ClasseFiglia : public ClassePadre { ... };
```

**⚠ IL TRABOCCHETTO:** Se ti dimentichi di scrivere la parola `public` (es. `class Auto : Veicolo`), il C++ imposterà l'ereditarietà come `private` di default. Questo renderà tutti i metodi del padre inaccessibili dal `main`, generando immediati errori di compilazione!

## 2. Come Funzionano i Costruttori (Inizializzazione a Cascata)

La classe figlia non eredita automaticamente i costruttori del padre. Deve richiamare esplicitamente il costruttore del padre nella sua **lista di inizializzazione** (prima di aprire le parentesi graffe del proprio costruttore).

```
Figlia(string n, int a) : Padre(n) {  
    // Qui inizializzi solo gli attributi esclusivi della figlia  
    attributo_figlia = a;  
}
```

*Logica temporale:* Quando crei un oggetto figlio, il computer esegue prima il costruttore del padre (crea la base) e solo dopo esegue il corpo del costruttore della figlia.

## 3. I Livelli di Accesso (Specificatori)

All'interno della classe Padre, la scelta della parola chiave cambia radicalmente ciò che la Figlia può vedere o modificare direttamente:

| Specificatore    | Accessibile dal <b>main</b> ?   | Accessibile dalle classi Figlie?                               |
|------------------|---------------------------------|----------------------------------------------------------------|
| <b>public</b>    | Sì, tramite l'oggetto.          | Sì, direttamente.                                              |
| <b>protected</b> | No (Garantisce incapsulamento). | <b>Sì</b> (Ideale per variabili scambiate tra padre e figlia). |
| <b>private</b>   | No.                             | No (La figlia deve usare i Getter/Setter pubblici del padre).  |

## 4. Struttura Modello Completa

Ecco lo schema pulito da usare come riferimento per non sbagliare la sintassi:

```
#include <iostream>
#include <string>
using namespace std;

class Veicolo {
protected:
    string targa; // Accessibile dalle figlie, protetto dal main

private:
    float prezzo; // Totalmente blindato

public:
    string marca;

    Veicolo(string m, float p, string t) {
        marca = m; prezzo = p; targa = t;
    }
    float get_prezzo() { return prezzo; }
};

// Ereditarietà pubblica dal padre
class Auto : public Veicolo {
public:
    int porte;

    // Il costruttore inoltra i dati al costruttore del Padre
    Auto(string m, float p, string t, int po) : Veicolo(m, p, t) {
        porte = po;
    }

    void mostra() {
        // targa è protected: la figlia la usa direttamente!
        cout << marca << " (" << targa << ") a " << porte << " porte." << endl;
        cout << "Prezzo: " << get_prezzo() << " euro." << endl;
    }
};
```

